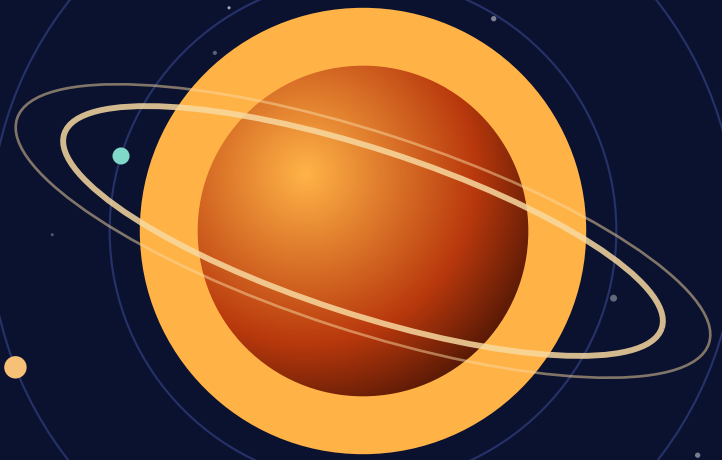
TALLER HANDS-ON · DATA ENGINEERING + DEVOPS

De los datos de la NASA a Docker Hub

Construye, contenieriza y publica un dashboard de exoplanetas con MySQL 8.4, Jupyter, Streamlit y Docker Compose.

Docker · MySQL 8.4 · Python 3.13 · Streamlit · Compose



Duración estimada: 4 h 10 min + pausas · Nivel: intermedio · 10 laboratorios guiados

INTRODUCCIÓN

Qué vamos a construir

AL TERMINAR EL TALLER SABRÁS

- Levantar un **entorno reproducible** con Docker: red, volumen, MySQL 8.4 y Jupyter.
- Hacer un **ETL** del catálogo de exoplanetas de la NASA hacia una base relacional.
- Modelar **4 tablas con claves foráneas** y verificar la integridad referencial.
- Ejecutar un **dashboard Streamlit** con 4 módulos de análisis astrofísico.
- **Empaquetarlo**: Dockerfile, docker-compose, healthchecks y variables de entorno.
- **Publicarlo en Docker Hub** y levantarlo desde cero con un solo comando.

> **Metodología.** Cada módulo sigue el mismo ritmo: **concepto → laboratorio guiado → checkpoint verificable**. No avances si tu checkpoint no está en verde.

RESULTADO FINAL

TERMINAL

```
$ docker compose up -d
 ✔ Container mysql84   Healthy
 ✔ Container app       Started

→ http://localhost:8501
```

6.375
exoplanetas confirmados

4.780
sistemas estelares

Transit
método de detección dominante

Cifras de la ejecución de referencia; el catálogo de la NASA se actualiza.

INTRODUCCIÓN

Agenda y prerrequisitos

AGENDA (250 MIN)

Bloque	Contenido	Min
0	Introducción y arquitectura	15
1	Datos de la NASA y modelo relacional	30
2	Lab 1–2 · Entorno Docker (red, MySQL, Jupyter)	30
3	Lab 3 · ETL hacia MySQL	45
4	Lab 4 · App Streamlit en local	30
5	Lab 5–9 · Contenerizar: Dockerfile y Compose	60
6	Lab 10 · Publicar en Docker Hub	30
+	Repaso, retos y cierre	10

Total ≈ 4 h 10 min, más pausas sugeridas tras los bloques 3 y 5.

ANTES DE EMPEZAR

- [ ] **Docker** Engine o Docker Desktop: `docker --version`
- [ ] Cuenta gratuita en **Docker Hub** (hub.docker.com)
- [ ] **Python 3.13** con `venv` en tu equipo (Lab 4)
- [ ] Navegador y, opcional, **DBeaver**
- [ ] Internet: API de la NASA y descarga de imágenes
- [ ] Espacio en disco: la imagen de Jupyter pesa varios GB
- [ ] Archivos: `1limpiar.sh`, `2crear.sh`, notebook ETL y `4astronomia_app.py`

INTRODUCCIÓN

Mapa del proyecto: el flujo actual

1 Limpiar

`1limpiar.sh`

Borra contenedores, imágenes y volúmenes

2 Crear

`2crear.sh`

Red + MySQL 8.4 + Jupyter

3 ETL

`notebook .ipynb`

NASA → 4 tablas → MySQL

4 App

`4astronomia_app.py`

Dashboard Streamlit en localhost

QUÉ QUEDA CORRIENDO TRAS LOS PASOS 2 Y 3



Conceptos Docker en una página

Imagen

Plantilla de solo lectura con tu app y sus dependencias. Se construye con un **Dockerfile**.

`docker build` · `docker images`
Analogía: la receta

Contenedor

Instancia en ejecución de una imagen. Es efímero: al borrarlo se pierde su disco interno.

`docker run` · `docker ps`
Analogía: el plato servido

Volumen

Almacenamiento persistente administrado por Docker. Sobrevive al contenedor.

`docker volume create`
Analogía: el disco externo

Red

Red virtual con DNS interno: los contenedores se encuentran por **nombre** (`mysql84`, `db`).

`docker network create`
Analogía: la agenda de contactos

Registry / Docker Hub

Almacén de imágenes: subes la tuya y cualquiera puede descargarla.

`docker push` · `docker pull`
Analogía: la biblioteca

Compose

Un archivo YAML que describe varios servicios, redes y volúmenes y los levanta juntos.

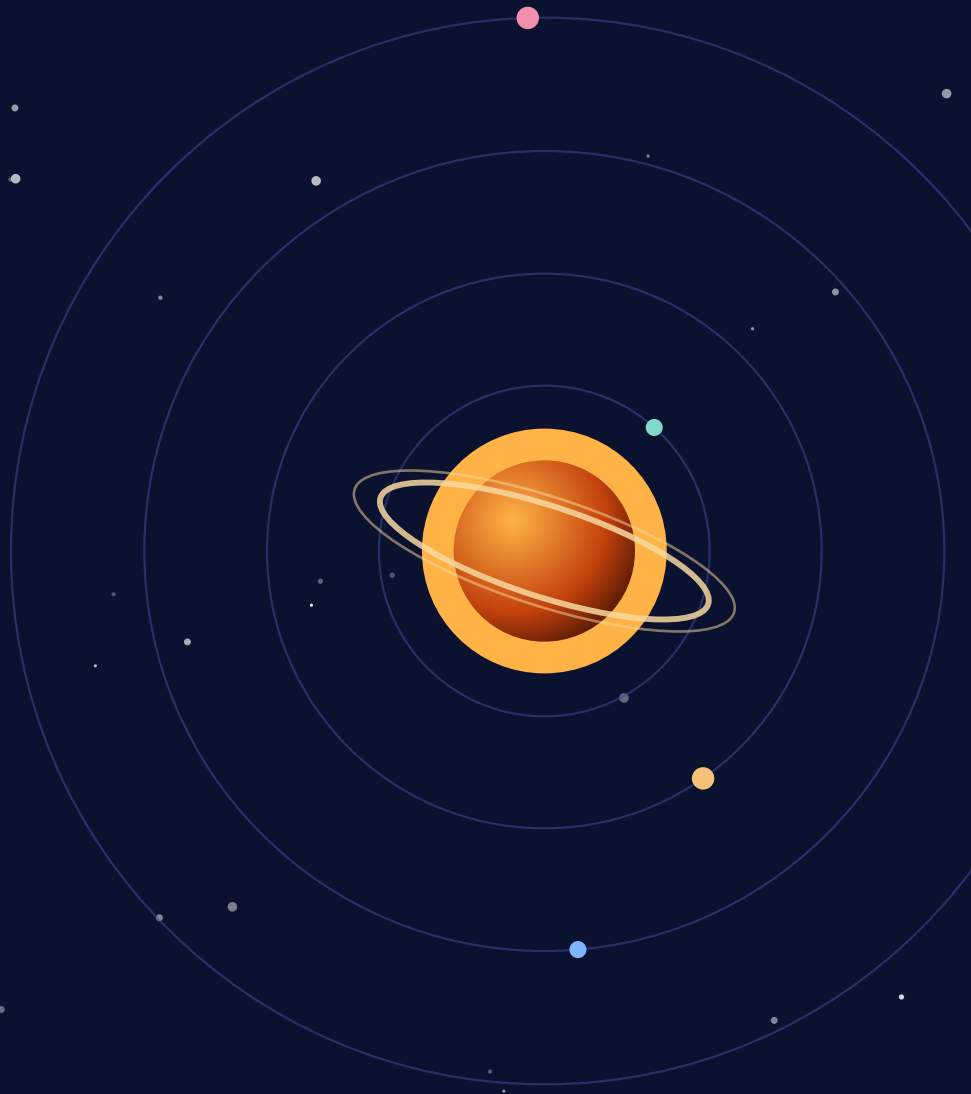
`docker compose up`
Analogía: el director de orquesta

MÓDULO 1

Datos y modelo relacional

NASA Exoplanet Archive → 4 tablas conectadas por claves foráneas.

01



Los datos: NASA Exoplanet Archive

Planeta

- `pl_name` nombre
- `pl_masse` masa (M⊕)
- `pl_rade` radio (R⊕)
- `pl_orbper` periodo (días)
- `pl_orbsmax` semieje (UA)

Estrella

- `hostname` nombre
- `st_mass` masa (M☉)
- `st_rad` radio (R☉)
- `st_teff` temperatura (K)
- `sy_snum` nº estrellas
- `sy_pnum` nº planetas

Descubrimiento

- `disc_year` año
- `discoverymethod` método
- `disc_facility` instalación
- `disc_telescope` telescopio
- `disc_instrument` instrumento

6.375
filas: un planeta por fila (pscomppars)

4.780
estrellas únicas (hostname)

≈ 61 %
planetas sin masa medida (3.908 NaN)

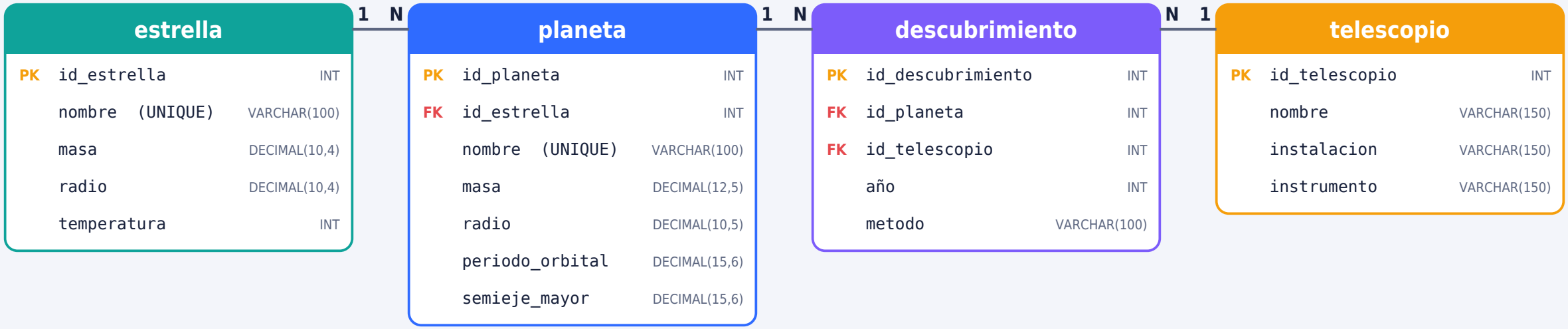
Datos reales, datos con huecos. El catálogo es real, por eso es **incompleto**: `pl_masse` falta en más de la mitad de los planetas. Los gráficos de masa–radio usan `dropna`; tú decides cómo tratar los vacíos.

ORIGEN

```
url = "https://exoplanetarchive.ipac.caltech.edu/TAP/sync"      # API TAP (Table Access Protocol)
params = {"query": "SELECT ... FROM pscomppars", "format": "csv"}
```

Cifras de la ejecución de referencia; el catálogo crece con nuevos descubrimientos.

Modelo relacional: 4 tablas



1 estrella → N planetas

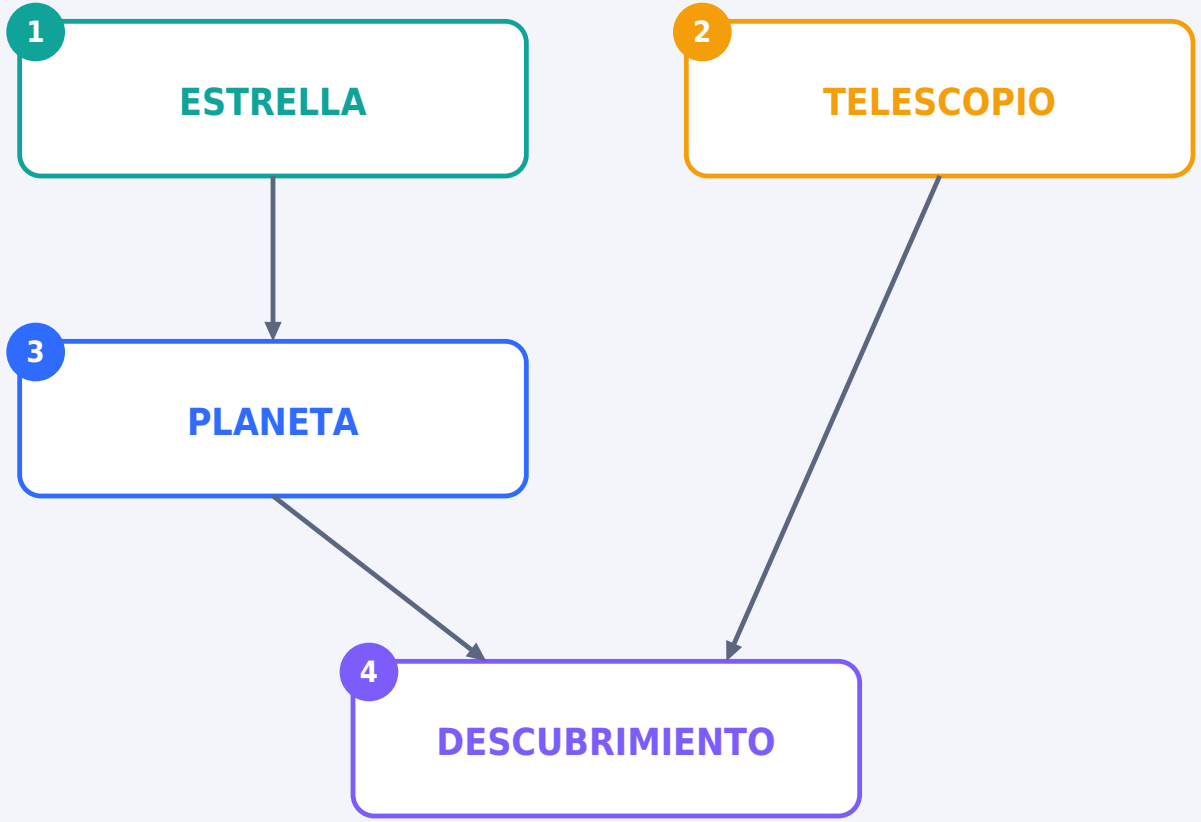
1 planeta → N descubrimientos (hoy 1:1)

1 telescopio → N descubrimientos

Ojo. El diagrama original del proyecto usaba tipos distintos (p. ej. `temperatura DECIMAL(8,2)`). En este taller la fuente de verdad es el **DDL que ejecuta el notebook**, mostrado arriba.

Predice. ¿Qué crees que pasará si insertas un planeta con `id_estrella = 99999`? Lo comprobarás en el checkpoint del Lab 3.

Integridad referencial: padres antes que hijos



1 y **2** son intercambiables. La flecha significa «el hijo apunta al padre» (FOREIGN KEY).

LA REGLA

- Una clave foránea solo puede apuntar a una fila que **ya existe** en la tabla padre.
- Por eso el orden de carga es **estrella → telescopio → planeta → descubrimiento**.
- Los IDs se generan en pandas (1..N) y se insertan explícitos; `AUTO_INCREMENT` continúa después.

SI TE SALTAS EL ORDEN

```mysql
ERROR 1452 (23000): Cannot add or update
a child row: a foreign key constraint fails
```

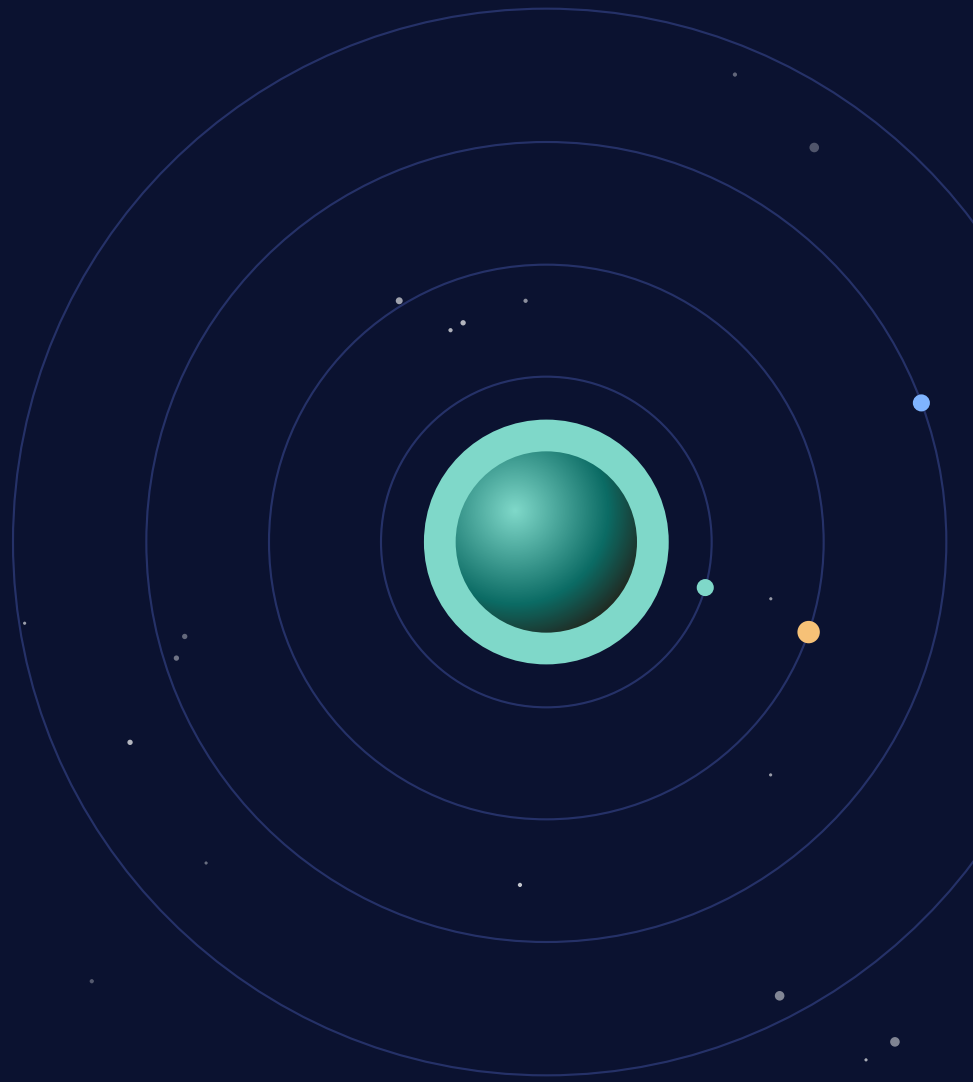
Truco. Al vaciar tablas se invierte el orden: **hijos primero** (descubrimiento → planeta → telescopio → estrella).

MÓDULO 2

Entorno Docker reproducible

Limpiar → crear red, volumen, MySQL 8.4 y Jupyter.

02



Limpiar solo lo que es tuyo

HANDS-ON · 5 min

Cuidado. El `1limpiar.sh` original ejecuta `docker system prune -a --volumes -f` y borra **todo** Docker del equipo, incluso proyectos ajenos. Úsalo solo en una máquina de laboratorio.

BASH

```
# Limpieza SOLO de este proyecto
sudo docker rm -f mysql84 jupyter 2>/dev/null
sudo docker volume rm mysql84-data 2>/dev/null
sudo docker network rm ciencia-net 2>/dev/null

# Verifica
sudo docker ps -a
sudo docker volume ls
sudo docker network ls
```

QUÉ HACE CADA LÍNEA

1. `rm -f` detiene y elimina solo los dos contenedores del proyecto.
2. `volume rm` borra la base: habrá que repetir el ETL (es intencional).
3. `network rm` elimina `ciencia-net`.
4. No tocamos las **imágenes**: evitamos re-descargar varios GB.

✓ **Checkpoint.** `docker ps -a` no muestra `mysql84` ni `jupyter`, y `docker volume ls` no lista `mysql84-data`.

Crear red, MySQL 8.4 y Jupyter

HANDS-ON · 15 min

BASH

```bash
# 1. Red y volúmenes
sudo docker network create ciencia-net
sudo docker volume create mysql84-data
sudo docker volume create jupyter-work

# 2. MySQL 8.4
sudo docker run -d --name mysql84 --network ciencia-net \
  -p 127.0.0.1:3306:3306 \
  -e MYSQL_ROOT_PASSWORD=root \
  -e MYSQL_DATABASE=astronomia \
  -v mysql84-data:/var/lib/mysql \
  mysql:8.4

# 3. Jupyter (con volumen para no perder tu trabajo)
sudo docker run -d --name jupyter --network ciencia-net \
  -p 8888:8888 \
  -v jupyter-work:/home/jovyan/work \
  quay.io/jupyter/scipy-notebook:latest
```

POR QUÉ ASÍ

- `--network ciencia-net`: ambos contenedores se ven por nombre.
- `127.0.0.1:3306:3306`: MySQL accesible solo desde tu equipo (DBeaver, app local).
- `-v mysql84-data`: la base sobrevive al contenedor.
- `jupyter-work`: tus notebooks y CSV persisten en `/home/jovyan/work`.
- `:latest` es cómodo en clase; en producción fija una versión.

> **Vs. el script.** `2crear.sh` hace lo mismo, más un `sleep 10`. Aquí añadimos el volumen de Jupyter y limitamos el puerto a 127.0.0.1.

Verificar y conectar (Jupyter y DBeaver)

HANDS-ON · 10 min

```bash
sudo docker ps

# MySQL listo cuando diga "ready for connections"
sudo docker logs mysql84 2>&1 | grep "ready for connections" | tail -1

# Enlace de Jupyter con su token
sudo docker logs jupyter 2>&1 | grep token | tail -1

# ¿Qué contenedores hay en la red?
sudo docker network inspect ciencia-net \
  --format '{{range .Containers}}{{.Name}}{{"\n"}}{{end}}'
```

Paciencia. En el primer arranque MySQL tarda en inicializar. Si `docker logs` aún no muestra «ready for connections», espera: por eso el script original dormía 10 s.

✓ **Checkpoint.** Abres Jupyter con el token y creas un notebook en `work/`. DBeaver conecta y ve la base `astronomia` (vacía).

UN MISMO MYSQL, DOS DIRECCIONES

Desde dónde	Host	Puerto
Jupyter (dentro de `ciencia-net`)	`mysql84`	3306
Tu equipo: DBeaver, app local	`localhost`	3306

Usuario `root` · clave `root` · base `astronomia`

Recuerda. `mysql84` solo existe dentro de la red Docker; `localhost` solo desde tu equipo. Esta idea es la clave del Lab 5.

Truco. DBeaver + MySQL 8: si aparece *Public Key Retrieval is not allowed*, añade `allowPublicKeyRetrieval=true` en las propiedades del driver.

MÓDULO 3

ETL hacia MySQL

Extraer de la NASA, transformar a 4 tablas y cargar respetando las claves foráneas.



03

Extraer: consultar la API de la NASA

HANDS-ON · 10 min

JUPYTER

```
# Celda 1
!pip install pandas requests mysql-connector-python sqlalchemy pymysql

# Celda 2
import requests, pandas as pd
from io import StringIO

query = """
SELECT pl_name, hostname, pl_masse, pl_rade, pl_orbper, pl_orbsmax,
       st_mass, st_rad, st_teff, sy_snum, sy_pnum,
       disc_year, discoverymethod, disc_facility,
       disc_telescope, disc_instrument
FROM pscomppars
"""
url = "https://exoplanetarchive.ipac.caltech.edu/TAP/sync"
r = requests.get(url, params={"query": query, "format": "csv"})
print(r.status_code)                    # 200
df = pd.read_csv(StringIO(r.text))
print(df.shape)                         # (6375, 16) aprox.
```

PASO A PASO

1. Abre Jupyter y crea `etl.ipynb` dentro de `work/`.
2. Ejecuta las celdas 1 y 2.
3. Explora: `df.info()` y `df.isna().sum()`.

PREGUNTAS

- ¿Cuántas estrellas únicas? `df['hostname'].nunique()`
- ¿Cuál tiene más planetas? `value_counts().head()` → KOI-351 (8)

✓ **Checkpoint.** Status 200 y `df.shape` ≈ (6375, 16). Tus cifras pueden variar: el catálogo crece.

Transformar I: estrella y planeta

HANDS-ON · 10 min

JUPYTER

```
# ESTRELLA: una fila por estrella (hostname)
estrellas = (df[["hostname", "st_mass", "st_rad", "st_teff"]]
             .drop_duplicates(subset=["hostname"])
             .reset_index(drop=True))
estrellas.insert(0, "id_estrella", range(1, len(estrellas) + 1))
estrellas = estrellas.rename(columns={"hostname": "nombre", "st_mass": "masa",
                                      "st_rad": "radio", "st_teff": "temperatura"})

# PLANETA: la FK id_estrella sale de un merge por nombre de estrella
planetas = df[["pl_name", "hostname", "pl_masse", "pl_rade",
               "pl_orbper", "pl_orbsmax"]].rename(columns={
    "pl_name": "nombre", "pl_masse": "masa", "pl_rade": "radio",
    "pl_orbper": "periodo_orbital", "pl_orbsmax": "semieje_mayor"})
planetas = planetas.merge(
    estrellas[["id_estrella", "nombre"]].rename(columns={"nombre": "hostname"}),
    on="hostname", how="left").drop(columns="hostname")
planetas.insert(0, "id_planeta", range(1, len(planetas) + 1))
```

IDEAS CLAVE

- `drop_duplicates`: 6.375 filas → 4.780 estrellas.
- `insert` crea el ID (1..N) que será la PK.
- `merge` traduce nombre de estrella → `id_estrella` (FK).
- Los `NaN` de pandas llegan como `NULL` a MySQL.

Ojo. `drop_duplicates` conserva la primera fila de cada estrella.

✓ **Checkpoint.** `len(estrellas)` → 4780
`planetas['id_estrella'].isna().sum()` → 0

Transformar II: telescopio y descubrimiento

HANDS-ON · 10 min

JUPYTER

```
# TELESCOPIO: combinaciones únicas instalación + telescopio + instrumento
telescopios = (df[["disc_facility", "disc_telescope", "disc_instrument"]]
               .drop_duplicates().reset_index(drop=True))
telescopios.columns = ["instalacion", "nombre", "instrumento"]
telescopios.insert(0, "id_telescopio", range(1, len(telescopios) + 1))

# DESCUBRIMIENTO: une planeta y telescopio por sus claves naturales
descubrimientos = (
    df[["pl_name", "disc_year", "discoverymethod", "disc_facility",
        "disc_telescope", "disc_instrument"]]
    .rename(columns={"pl_name": "nombre_planeta", "disc_year": "año",
                     "discoverymethod": "metodo", "disc_facility": "instalacion",
                     "disc_telescope": "telescopio",
                     "disc_instrument": "instrumento"})
    .merge(planetas[["id_planeta", "nombre"]],
           left_on="nombre_planeta", right_on="nombre", how="left")
    .merge(telescopios.rename(columns={"nombre": "telescopio"}),
           on=["instalacion", "telescopio", "instrumento"], how="left"))
descubrimientos.insert(0, "id_descubrimiento", range(1, len(descubrimientos) + 1))
descubrimientos = descubrimientos[["id_descubrimiento", "id_planeta",
                                   "id_telescopio", "año", "metodo"]]
```

IDEAS CLAVE

- Un telescopio es la combinación de **3 campos**; por eso el merge usa 3 claves.
- En la tabla se llama `nombre`; se renombra solo para el merge.
- Al final solo quedan las 5 columnas del DDL.

> ✓ **Checkpoint.** `len(descubrimientos) == len(df)` → True
> Sin nulos en `id_planeta` ni `id_telescopio`.

Cargar: crear la base y poblar las tablas

HANDS-ON · 10 min

1 · CREAR

```
import mysql.connector
cn = mysql.connector.connect(host="mysql84", user="root", password="root")
cur = cn.cursor()
cur.execute("CREATE DATABASE IF NOT EXISTS astronomia")
cur.execute("USE astronomia")
# CREATE TABLE IF NOT EXISTS estrella (...), telescopio (...),
# planeta (...), descubrimiento (...)  → DDL completo en el Apéndice A
cn.close()
```

2 · CARGAR

```
from sqlalchemy import create_engine
engine = create_engine("mysql+pymysql://root:root@mysql84:3306/astronomia")

# Orden: padres primero, hijos después
estrellas.to_sql("estrella", engine, if_exists="append", index=False)
telescopios.to_sql("telescopio", engine, if_exists="append", index=False)
planetas.to_sql("planeta", engine, if_exists="append", index=False)
descubrimientos.to_sql("descubrimiento", engine, if_exists="append", index=False)
```

POR QUÉ ASÍ

- `append` conserva las PK/FK del DDL; `replace` las destruiría.
- El host es `mysql84`: Jupyter está dentro de la red.
- Si PyMySQL falla con `caching_sha2_password`, instala `cryptography`.

> **Nota.** Las 4 celdas terminan sin error. Si repites la carga fallará por claves duplicadas: es normal (siguiente diapositiva).

Checkpoint: verifica la base con SQL

HANDS-ON · 5 min

DBEAVER / MYSQL

```sql
-- 1. Conteo por tabla
SELECT 'estrella' AS tabla, COUNT(*) AS filas FROM estrella
UNION ALL SELECT 'planeta', COUNT(*) FROM planeta
UNION ALL SELECT 'telescopio', COUNT(*) FROM telescopio
UNION ALL SELECT 'descubrimiento', COUNT(*) FROM descubrimiento;

-- 2. Top métodos de detección
SELECT metodo, COUNT(*) AS total
FROM descubrimiento GROUP BY metodo ORDER BY total DESC LIMIT 5;

-- 3. ¿Funciona la clave foránea? (debe FALLAR)
INSERT INTO planeta (id_estrella, nombre) VALUES (99999, 'Prueba b');
```

RESULTADO ESPERADO

Tabla	Filas
estrella	4.780
planeta	6.375
telescopio	= `len(telescopios)`
descubrimiento	6.375

Cifras de la ejecución de referencia.

Consulta 2. *Transit* debe ocupar el primer lugar.

Consulta 3. Debe aparecer **ERROR 1452**: MySQL protege la integridad. Respondiste la pregunta de la diapositiva del modelo.

Errores típicos del ETL

Síntoma	Causa	Solución
`Duplicate entry '…' for key 'planeta.nombre'`	Ejecutaste la carga dos veces (`append`).	Vaciar las tablas (hijos → padres) y repetir.
`1452 Cannot add or update a child row`	Orden de inserción incorrecto.	Estrella, telescopio, planeta, descubrimiento.
`Can't connect to MySQL server on 'mysql84'`	MySQL aún arranca o está en otra red.	`docker logs mysql84`; revisa `docker network inspect ciencia-net`.
Error de conexión al llamar a la NASA	Sin internet en el contenedor o servicio caído.	Reintenta; revisa proxy/red de Docker.
Desaparecen CSV y notebooks	Se borró el contenedor sin volumen.	Trabaja en `/home/jovyan/work` (volumen `jupyter-work`).

REINICIAR LAS TABLAS SIN RECREAR LA BASE (ETL REPETIBLE)

```sql
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE descubrimiento; TRUNCATE planeta; TRUNCATE telescopio; TRUNCATE estrella;
SET FOREIGN_KEY_CHECKS = 1;
```

MÓDULO 4

La aplicación Streamlit

Un dashboard de 4 módulos que vive de una sola consulta a MySQL.

04



Anatomía de la app

1 Configuración
`st.set_page_config` · layout wide

↓

2 Conexión
`init_connection()` · `@st.cache_resource`

↓

3 Carga
`load_master_data()` · `@st.cache_data` → `df_master`

↓

4 Filtros
Slider de año + multiselect de método → `df_filtrado`

↓

5 Módulos
Radio en la barra lateral: 4 vistas

UNA CONSULTA, CUATRO JOIN

SQL

```sql
SELECT p.nombre AS planeta, p.masa AS p_masa, p.radio AS p_radio,
       p.periodo_orbital, p.semieje_mayor,
       e.nombre AS estrella, e.masa AS e_masa, e.radio AS e_radio,
       e.temperatura,
       d.año, d.metodo,
       t.nombre AS telescopio, t.instalacion, t.instrumento
FROM planeta p
JOIN estrella e       ON p.id_estrella = e.id_estrella
JOIN descubrimiento d ON p.id_planeta  = d.id_planeta
JOIN telescopio t     ON d.id_telescopio = t.id_telescopio
```

Idea. Toda la app vive de un solo DataFrame: `df_master` → `df_filtrado`. Los JOIN reconstruyen el modelo en una tabla plana de ≈ 6.375 filas.

Ojo. Son **INNER JOIN**: un planeta sin telescopio asociado desaparecería del dashboard.

Los 4 módulos de análisis

Visión General

KPIs: exoplanetas, sistemas, telescopios y método dominante. Histograma apilado por año y técnica.

Pregunta: ¿Cómo cambió la forma de descubrir planetas?

Astrofísica Planetaria

Masa–radio en escala log–log (supertierras vs gigantes gaseosos) y tercera ley de Kepler: log P vs log a con pendiente 3/2.

Pregunta: ¿Obedecen los exoplanetas la mecánica clásica?

Análisis Estelar

Boxplot de temperatura por método y aproximación al diagrama H-R (radio vs temperatura, eje invertido).

Pregunta: ¿Qué estrellas favorece cada técnica?

Instrumentación y Métodos

Treemap del Top 15 de instalaciones y telescopios por número de descubrimientos.

Pregunta: ¿Qué telescopios concentran los hallazgos?

Ejecutar la app original en tu equipo

HANDS-ON · 15 min

```bash
# En la carpeta del proyecto (tu equipo, NO en Docker)
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install streamlit pandas numpy plotly mysql-connector-python
streamlit run 4astronomia_app.py
# → http://localhost:8501
```

Por qué funciona. La app original se conecta a `localhost:3306` con `root/root`. Funciona porque `2crear.sh` publicó el puerto 3306 en tu equipo.

Rompe a propósito. Detén MySQL (`docker stop mysql84`), cierra la app (Ctrl+C) y vuelve a lanzarla: verás un **traceback** de conexión. Es la falta de manejo de errores que arreglaremos en el Lab 5. Después: `docker start mysql84`.

QUÉ OBSERVAR

1. La métrica **Exoplanetas Confirmados** muestra 6,375.
2. Mueve el slider de años: el conteo cambia.
3. Deja solo *Radial Velocity* en el multiselect.
4. Recorre los 4 módulos del menú lateral.

✓ **Checkpoint.** Ves los 4 módulos con datos y los filtros afectan a todos los gráficos.

MÓDULO 5

Contenerizar la aplicación

Refactor, Dockerfile, docker-compose y prueba desde cero.

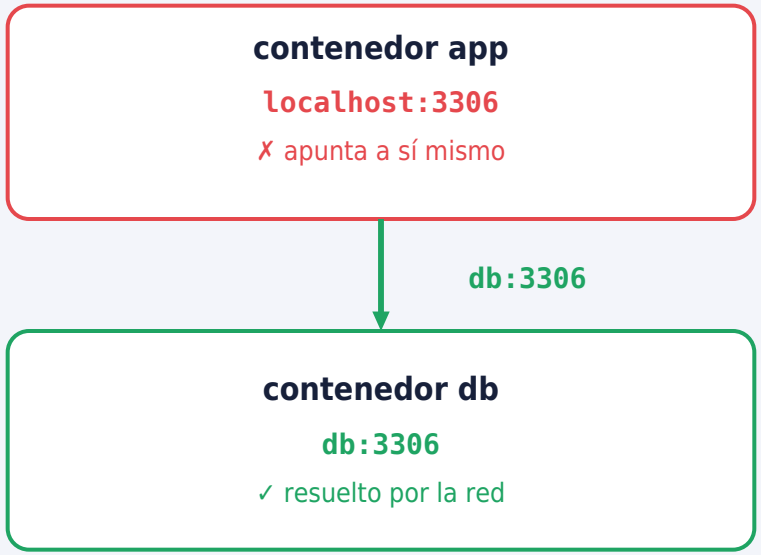
05



Por qué la app no está lista para Docker

Problema	Síntoma	Solución
`host="localhost"`	`Can't connect to MySQL server on 'localhost:3306'`	Host por variable de entorno (`DB_HOST=db`)
Credenciales en el código	Secretos visibles en una imagen pública	Variables de entorno y `.env`
Base vacía al arrancar	`ValueError: cannot convert float NaN to integer`	Validar `df.empty` y `st.stop()`
Conexión única cacheada	Falla tras inactividad del servidor	SQLAlchemy con `pool_pre_ping`
`use_container_width`	Deprecado: se retirará en una versión futura	`width="stretch"`
Strings LaTeX sin `r"…"`	`SyntaxWarning` en Python 3.12+	Raw strings

IDEA CLAVE



Recuerda. Dentro de un contenedor, `localhost` es el propio contenedor. La app debe llegar a MySQL por el **nombre del servicio**.

Refactor: configuración por entorno y errores claros

HANDS-ON · 10 min

APP.PY · 1/2

```python
import os
import pandas as pd
import streamlit as st
from sqlalchemy import create_engine
from sqlalchemy.engine import URL

QUERY = """SELECT ...  -- tu consulta con 4 JOIN"""

@st.cache_resource
def get_engine():
    url = URL.create(
        "mysql+mysqlconnector",
        username=os.environ["DB_USER"],
        password=os.environ["DB_PASSWORD"],
        host=os.getenv("DB_HOST", "localhost"),
        port=int(os.getenv("DB_PORT", "3306")),
        database=os.getenv("DB_NAME", "astronomia"))
    return create_engine(url, pool_pre_ping=True)
```

Además. Cambia los 6 `use_container_width=True` por `width="stretch"` y antepón `r` a los strings con LaTeX (en raw string usa `\` en vez de `\\`). `URL.create` escapa bien las contraseñas con caracteres especiales.

APP.PY · 2/2

```python
@st.cache_data(ttl=3600)
def load_master_data():
    return pd.read_sql(QUERY, get_engine())

try:
    df_master = load_master_data()
except Exception as e:
    st.error(f"No se pudo conectar: {e}")
    st.stop()

if df_master.empty:
    st.warning("La base está vacía.")
    st.stop()
```

✓ **Checkpoint.** `DB_USER=root DB_PASSWORD=root streamlit run app.py` sigue funcionando en local. Con MySQL detenido, la app muestra un mensaje de error en vez de un traceback.

Dependencias, .dockerignore y estructura

HANDS-ON · 5 min

BASH

```bash
# Saca las versiones que YA te funcionan
pip freeze | grep -iE "streamlit|pandas|numpy|plotly|sqlalchemy|mysql"
```

REQUIREMENTS.TXT

```
# requirements.txt
streamlit==X.Y.Z
pandas==X.Y.Z
numpy==X.Y.Z
plotly==X.Y.Z
sqlalchemy==X.Y.Z
mysql-connector-python==X.Y.Z
```

.DOCKERIGNORE

```
# .dockerignore
*.csv
*.ipynb
*.sh
.git
.env
__pycache__
```

ESTRUCTURA DEL PROYECTO

ÁRBOL

```
astronomia/
├── app.py              ← 4astronomia_app.py renombrado
├── requirements.txt
├── Dockerfile
├── .dockerignore
├── docker-compose.yml
├── init.sql            ← dump de MySQL (Lab 8)
└── .env                ← claves (NO se sube)
```

¿Por qué fijar versiones? Streamlit retira APIs deprecadas entre versiones. Con versiones fijas, la imagen de hoy se construye igual dentro de un año.

¿Y el .dockerignore? Deja fuera CSV, notebooks, scripts y `.env`: la imagen es más pequeña y no filtra secretos.

Anatomía del Dockerfile

HANDS-ON · 10 min

DOCKERFILE

```dockerfile
FROM python:3.13-slim

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY 4astronomia_app.py app.py

RUN useradd -m appuser
USER appuser

EXPOSE 8501
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s \
  CMD python -c "import urllib.request as u; u.urlopen('http://localhost:8501/_stcore/health')" || exit 1

CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0", \
     "--server.port=8501", "--server.headless=true"]
```

✓ **Checkpoint.** `docker build -t TUUSUARIO/astronomia-app:dev .` termina sin errores y `docker images` la lista.

POR QUÉ CADA DECISIÓN

- **FROM**: misma versión que el notebook; `slim` = ligera.
- **COPY requirements** antes del código: la capa de pip queda en caché.
- **--no-cache-dir**: imagen más pequeña.
- **USER**: no ejecutar como root.
- **EXPOSE**: documenta el puerto, no lo publica.
- **HEALTHCHECK**: Streamlit expone `/_stcore/health`.
- **--server.address=0.0.0.0**: sin esto no se ve desde fuera.

Construir y ejecutar en la red existente

HANDS-ON · 10 min

```bash
# Construir (reemplaza TUUSUARIO por tu usuario de Docker Hub)
docker build -t TUUSUARIO/astronomia-app:dev .

# Ejecutar en la MISMA red que MySQL
docker run --rm --name astro-app -p 8501:8501 \
  --network ciencia-net \
  -e DB_HOST=mysql84 -e DB_USER=root \
  -e DB_PASSWORD=root -e DB_NAME=astronomia \
  TUUSUARIO/astronomia-app:dev

# En otra terminal
docker ps
docker logs -f astro-app
```

EXPERIMENTOS

1. Abre `http://localhost:8501`: ya corre **desde un contenedor**.
2. Rompe a propósito: repite con `-e DB_HOST=localhost` y observa el mensaje de error. ¿Por qué?
3. Repite con `DB_HOST=mysql84` pero **sin** `--network`: el nombre no se resuelve.

> ✓ **Checkpoint.** El dashboard se ve igual que en el Lab 4, pero ahora corre en un contenedor y llega a MySQL por nombre.

Una base portátil: init.sql y .env

HANDS-ON · 5 min

Orden importante. Haz el volcado **antes** de borrar el volumen `mysql84-data`: después ya no habrá datos que volcar.

BASH

```bash
docker exec mysql84 mysqldump -uroot -proot \
  --databases astronomia --default-character-set=utf8mb4 \
  --single-transaction > init.sql

ls -lh init.sql
grep -c "INSERT INTO" init.sql      # ≥ 4 (al menos una por tabla)
```

.ENV

```
# .env  (NO se sube a Git ni a la imagen)
MYSQL_ROOT_PASSWORD=cambia_esta_clave
MYSQL_PASSWORD=otra_clave_distinta
```

DOS FORMAS DE ENTREGAR LOS DATOS

- **A · Montar** `init.sql` en el contenedor de MySQL (lo hará Compose).
- **B · Hornear** los datos en una imagen propia `astronomia-db`.

DOCKERFILE.DB

```
# Dockerfile.db (opción B)
FROM mysql:8.4
COPY init.sql /docker-entrypoint-initdb.d/
```

Importante. MySQL ejecuta lo que haya en `/docker-entrypoint-initdb.d/` **solo en el primer arranque** (volumen vacío).

✓ **Checkpoint.** `init.sql` contiene `CREATE TABLE` e `INSERT INTO` de las 4 tablas.

docker-compose.yml: base + app

HANDS-ON · 10 min

DOCKER-COMPOSE.YML · 1/2

```yaml
services:
  db:
    image: mysql:8.4
    container_name: mysql84
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: astronomia
      MYSQL_USER: astro
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    volumes:
      - mysql84-data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "--silent"]
      interval: 10s
      retries: 10
      start_period: 30s
    networks: [ciencia-net]
```

DOCKER-COMPOSE.YML · 2/2

```yaml
  app:
    build: .
    image: TUUSUARIO/astronomia-app:1.0.0
    ports:
      - "8501:8501"
    environment:
      DB_HOST: db
      DB_USER: astro
      DB_PASSWORD: ${MYSQL_PASSWORD}
      DB_NAME: astronomia
    depends_on:
      db:
        condition: service_healthy
    networks: [ciencia-net]

volumes:
  mysql84-data:

networks:
  ciencia-net:
```

healthcheck en 127.0.0.1

Durante la inicialización MySQL solo escucha por socket; con `localhost` el check daría «listo» antes de tiempo.

depends_on + healthy

La app arranca cuando la base responde, no solo cuando su contenedor existe.

DB_HOST: db

Nombre del servicio = nombre DNS dentro de la red. El usuario `astro` solo ve la base `astronomia`.

Prueba desde cero

HANDS-ON · 5 min

```bash
# 0. Libera nombres y puertos de los contenedores manuales
docker rm -f mysql84 jupyter

# 1. Arranque limpio
docker compose down -v          # borra los volúmenes del proyecto
docker compose up --build -d

# 2. Observa
docker compose ps               # db: healthy · app: running
docker compose logs -f app      # Ctrl+C para salir
```

Recuerda. `init.sql` solo corre con el volumen vacío. Si cambias el volcado, haz `docker compose down -v` para reinicializar.

Nota. Jupyter ya no hace falta para **ejecutar** la app; solo para regenerar los datos desde la NASA.

CHECKLIST DE ACEPTACIÓN

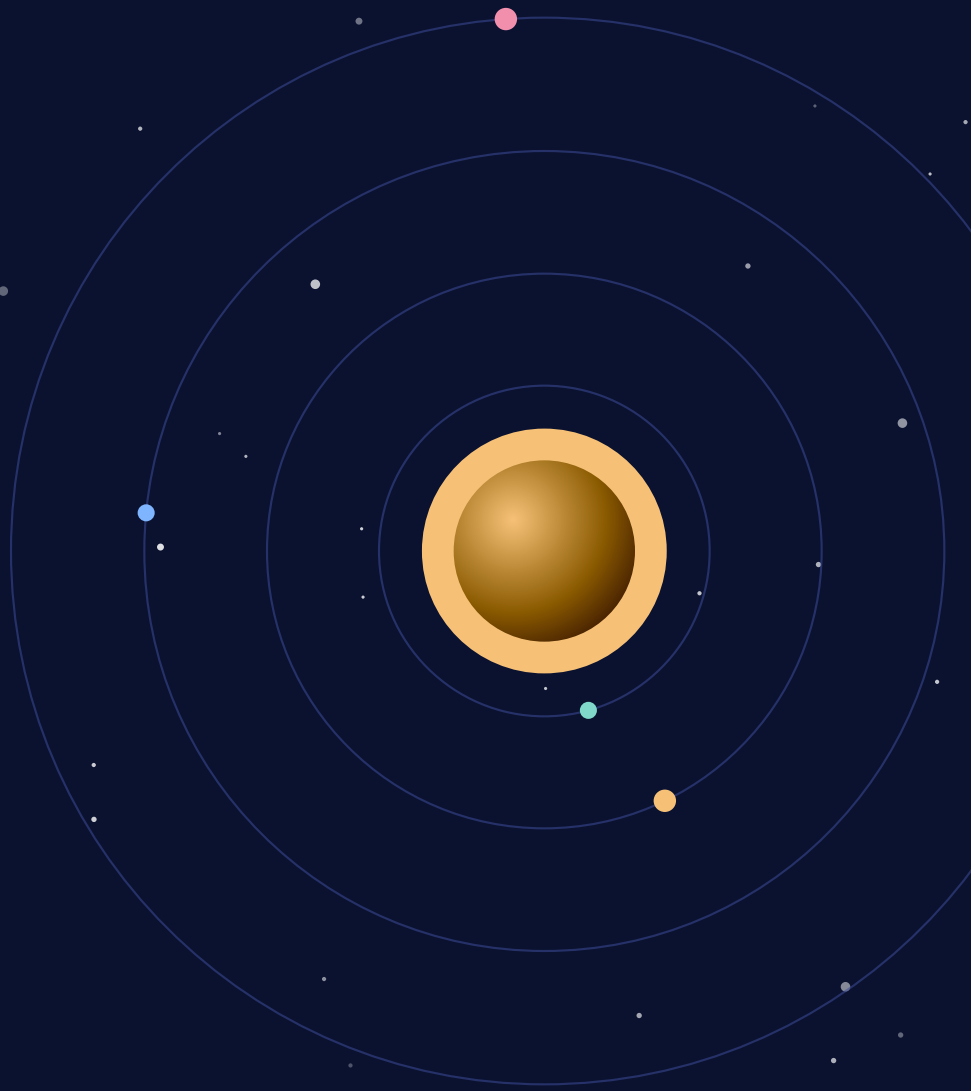
- [ ] `db` aparece como **healthy**
- [ ] `app` aparece como **running**
- [ ] El dashboard en `localhost:8501` muestra ≈ 6.375 exoplanetas
- [ ] Repites `down -v` y `up`: queda igual

MÓDULO 6

Publicar en Docker Hub

Etiquetar, subir y verificar tu imagen como lo haría otra persona.

06



Etiquetar, subir y verificar

HANDS-ON · 20 min

BASH

```bash
# 1. Autenticación (con 2FA usa un access token)
docker login -u TUUSUARIO

# 2. Etiquetas: versión fija + latest
docker build -t TUUSUARIO/astronomia-app:1.0.0 \
             -t TUUSUARIO/astronomia-app:latest .

# 3. Subir
docker push TUUSUARIO/astronomia-app:1.0.0
docker push TUUSUARIO/astronomia-app:latest
```

BASH

```bash
# 4. Verifica como si fueras otra persona
docker compose down -v
docker rmi TUUSUARIO/astronomia-app:1.0.0
# En docker-compose.yml comenta la línea "build: ."
docker compose up -d     # ← descarga la imagen desde Docker Hub
```

OPCIONAL

MULTI-ARCH

```bash
# Imagen para amd64 y arm64 (Mac M1/M2/M3, Raspberry Pi)
docker buildx build --platform linux/amd64,linux/arm64 \
  -t TUUSUARIO/astronomia-app:1.0.0 --push .
# si falla: docker buildx create --use
```

DB

```bash
# Imagen de la base con los datos incluidos
docker build -f Dockerfile.db -t TUUSUARIO/astronomia-db:1.0.0 .
docker push TUUSUARIO/astronomia-db:1.0.0
```

Nombre. `usuario/repositorio:tag`. Sin el prefijo `TUUSUARIO/` el push falla con «access denied».

✓ **Checkpoint.** Tu imagen aparece en hub.docker.com y Compose la descarga sin construirla.

Antes del push: seguridad y buenas prácticas

CHECKLIST PREVIO AL PUSH

- [ ] Sin contraseñas, tokens ni `.env` dentro de la imagen.
- [ ] `.dockerignore` excluye CSV, notebooks y `.git`.
- [ ] Dependencias fijadas en `requirements.txt`.
- [ ] Proceso como **usuario no root**.
- [ ] `HEALTHCHECK` definido.
- [ ] Tag de versión (`1.0.0`) además de `latest`.
- [ ] Probado con `down -v` y luego `up`, desde cero.

SOBRE DOCKER HUB

- La cuenta **Personal gratuita** incluye repositorios públicos ilimitados y 1 repositorio privado.
- Las descargas tienen **límites de tasa**; consulta las cifras vigentes en docs.docker.com/docker-hub.
- Todo repo público lo lee cualquiera: `docker history` muestra las capas.
- Escribe un **README**: qué hace, variables de entorno y un ejemplo de Compose.
- Cita la fuente de los datos: NASA Exoplanet Archive.

BASH

```bash
# Opcional: revisar vulnerabilidades
docker scout quickview TUUSUARIO/astronomia-app:1.0.0
```

REFERENCIA

Troubleshooting: síntoma → causa → solución

Síntoma	Causa probable	Solución
La app abre pero dice «Can't connect to MySQL server»	`DB_HOST` incorrecto o `db` en otra red	Usa el nombre del servicio (`db`) y revisa `docker compose ps`
No abre en el navegador	Falta `--server.address=0.0.0.0` o `-p 8501:8501`	Corrige el `CMD` y publica el puerto
Las tablas están vacías tras el `up`	El volumen ya existía: `init.sql` no se ejecutó	`docker compose down -v` y volver a levantar
`exec format error`	Arquitectura distinta (ARM vs amd64)	Construye multi-arch con `buildx`
`denied: requested access to the resource is denied`	Imagen sin prefijo de usuario o sin login	`docker login` y etiqueta `USUARIO/repo:tag`
DBeaver: *Public Key Retrieval is not allowed*	Autenticación `caching_sha2_password`	`allowPublicKeyRetrieval=true` en el driver
`port is already allocated` (3306)	Hay otro MySQL usando el puerto	Cambia el mapeo a `3307:3306` o detén el otro

REFERENCIA

Chuleta de comandos Docker

IMÁGENES

```
docker build -t USER/app:1.0.0 .
docker images
docker tag ORIGEN DESTINO
docker push USER/app:1.0.0
docker pull USER/app:1.0.0
docker history USER/app:1.0.0
docker rmi USER/app:1.0.0
```

CONTENEDORES

```
docker ps -a
docker logs -f NOMBRE
docker exec -it NOMBRE bash
docker stop NOMBRE
docker start NOMBRE
docker rm -f NOMBRE
docker run --rm -p 8501:8501 IMG
docker inspect NOMBRE
```

COMPOSE, DATOS Y RED

```
docker compose up -d --build
docker compose ps
docker compose logs -f app
docker compose down      # conserva
docker compose down -v   # borra vol.
docker volume ls
docker network ls
docker network inspect NET
```

FLUJO DE DEPURACIÓN CUANDO ALGO FALLA

1. **`docker ps -a`** — ¿está corriendo?
2. **`docker logs -f X`** — ¿qué error muestra?
3. **`docker exec -it X bash`** — entra y comprueba
4. **`docker inspect X`** — red, env y volúmenes

CIERRE

Retos para seguir practicando

Dificultad ★

Filtro por temperatura estelar

Añade un slider de `temperatura` al sidebar y comprueba que todos los gráficos respondan.

Dificultad ★★

Jupyter dentro de Compose

Agrega un servicio `jupyter` con `profiles: [dev]` para tener ETL y app en un solo YAML.

Dificultad ★★

ETL como script

Convierte el notebook en `etl.py` repetible y ejecútalo como servicio de un solo uso.

Dificultad ★★

Usuario de solo lectura

Crea un usuario con `GRANT SELECT` para la app y deja `root` solo para administrar.

Dificultad ★★★

Publicación automática

GitHub Actions: build y push de la imagen al crear un tag `v*`.

Dificultad ★★★

Multi-arquitectura

Publica para amd64 y arm64 con `buildx` y verifícalo en otra máquina.

CIERRE

Repaso: 6 preguntas

1. ¿Por qué `localhost` falla dentro de un contenedor cuando MySQL está en otro?
2. ¿Por qué se inserta **estrella** antes que **planeta**?
3. ¿Qué diferencia hay entre una **imagen** y un **contenedor**?
4. ¿Qué hace `depends_on` con `condition: service_healthy`?
5. ¿Cuándo se ejecutan los scripts de `/docker-entrypoint-initdb.d/`?
6. ¿Por qué no debe ir el `.env` dentro de la imagen que subes a Docker Hub?

Sugerencia: resuélvelas en parejas y compara antes de ver el solucionario.

CIERRE

Solucionario (instructor)

1. Cada contenedor tiene su propia red: `localhost` es él mismo. Hay que usar el nombre del servicio (`db`) dentro de la red.

2. Por la integridad referencial: una FK solo puede apuntar a una fila que ya existe en la tabla padre (error 1452 si no).

3. La imagen es la plantilla inmutable; el contenedor es una instancia en ejecución, con su capa de escritura efímera.

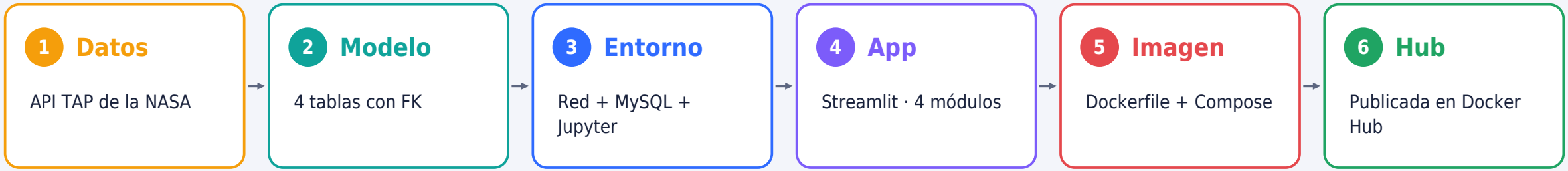
4. Compose espera a que el healthcheck de `db` sea correcto antes de arrancar `app`, no solo a que el contenedor exista.

5. Solo en el **primer arranque**, con el directorio de datos vacío. Con volumen ya poblado se ignoran.

6. Las capas de una imagen pública son legibles por cualquiera (`docker history`): expondría las contraseñas.

CIERRE

Lo que construiste



QUÉ TE LLEVAS

- Un proyecto **reproducible**: cualquiera lo levanta con `docker compose up`.
- El hábito de **verificar** con checkpoints en cada paso.
- Separar **configuración** (entorno) de **código** (imagen).

SIGUIENTES PASOS

- Automatizar build y push con CI/CD.
- Añadir tests y análisis de vulnerabilidades a la imagen.
- Probar otros orquestadores cuando necesites escalar.

Fuente. Datos: NASA Exoplanet Archive. Los números del taller provienen de la ejecución de referencia del notebook y pueden variar con las actualizaciones del catálogo.

APÉNDICE A

DDL completo de la base astronomia

SQL · 1/2

```sql
CREATE DATABASE IF NOT EXISTS astronomia;
USE astronomia;

CREATE TABLE IF NOT EXISTS estrella (
  id_estrella INT AUTO_INCREMENT,
  nombre VARCHAR(100) NOT NULL,
  masa DECIMAL(10,4),
  radio DECIMAL(10,4),
  temperatura INT,
  PRIMARY KEY (id_estrella),
  UNIQUE (nombre)
);

CREATE TABLE IF NOT EXISTS telescopio (
  id_telescopio INT AUTO_INCREMENT,
  nombre VARCHAR(150),
  instalacion VARCHAR(150),
  instrumento VARCHAR(150),
  PRIMARY KEY (id_telescopio)
);
```

SQL · 2/2

```sql
CREATE TABLE IF NOT EXISTS planeta (
  id_planeta INT AUTO_INCREMENT,
  id_estrella INT NOT NULL,
  nombre VARCHAR(100) NOT NULL,
  masa DECIMAL(12,5),
  radio DECIMAL(10,5),
  periodo_orbital DECIMAL(15,6),
  semieje_mayor DECIMAL(15,6),
  PRIMARY KEY (id_planeta),
  UNIQUE (nombre),
  FOREIGN KEY (id_estrella) REFERENCES estrella(id_estrella)
);

CREATE TABLE IF NOT EXISTS descubrimiento (
  id_descubrimiento INT AUTO_INCREMENT,
  id_planeta INT NOT NULL,
  id_telescopio INT,
  año INT,
  metodo VARCHAR(100),
  PRIMARY KEY (id_descubrimiento),
  FOREIGN KEY (id_planeta) REFERENCES planeta(id_planeta),
  FOREIGN KEY (id_telescopio) REFERENCES telescopio(id_telescopio)
);
```